

COMPLEMENTO MANUAL TÉCNICO - EXPLICACIONES DE CÓDIGO

NIVEL 1: CIMIENTOS

3.1 `app/utils/logger.py`

Explicación del Código:

```
def __init__(self, name: str, level: str = "INFO"):
    self.logger = logging.getLogger(name)
    self.logger.setLevel(getattr(logging, level))
```

- `logging.getLogger(name)`: Crea logger con identificador único por clase
- `getattr(logging, level)`: Convierte string "INFO" a constante logging.INFO dinámicamente

```
handler = logging.StreamHandler()
handler.setFormatter(logging.Formatter('%(message)s'))
self.logger.addHandler(handler)
```

- `StreamHandler()`: Envía logs a stdout (consola)
- `Formatter('%(message)s')`: Solo imprime mensaje, sin formato adicional (lo manejamos en JSON)

```
def _format_message(self, level: str, message: str, **kwargs) -> str:
    log_entry = {
        "timestamp": datetime.now().isoformat(),
        "level": level,
        "message": message,
        **kwargs
    }
    return json.dumps(log_entry)
```

- `**kwargs`: Permite pasar contexto adicional (`series="PD04637PD"`, `records=85`)
- `json.dumps()`: Serializa dict a string JSON
- Resultado: Log estructurado parseable por herramientas

3.2 `app/utils/dates.py`

Explicación del Código:

```
def get_date_range(days_back: int = 30) -> Tuple[str, str]:
    end_date = datetime.now()
    start_date = end_date - timedelta(days=days_back)
```

- `timedelta(days=days_back)` : Resta días a fecha actual
- Ejemplo: Si hoy es 2026-05-01 y `days_back=30` ---> `start_date = 2026-04-01`

```
return (
    start_date.strftime("%Y-%m-%d"),
    end_date.strftime("%Y-%m-%d")
)
```

- `%-m` y `%-d` : Elimina padding de ceros (Linux/Mac)
- Formato BCR requiere "2026-4-1" no "2026-04-01"
- En Windows usar `%#m` y `%#d`

```
def get_partition_path(date: datetime) -> str:
    return date.strftime("%Y/%m/%d")
```

- Con ceros para estructura de directorios consistente
- Facilita ordenamiento y navegación filesystem

3.3 `app/config/settings.py`

Explicación del Código:

```
load_dotenv()
```

- Carga archivo `.env` a variables de entorno `os.environ`
- Se ejecuta al importar el módulo

```
def _load_config(self):
    with open(Path("config.yaml"), 'r') as f:
        return yaml.safe_load(f)
```

- `yaml.safe_load()` : Parsea YAML a dict Python
- `safe_load()` : Previene ejecución de código arbitrario

```
@property
def bcr_base_url(self) -> str:
    return os.getenv("BCR_BASE_URL", self.config['api']['base_url'])
```

- `@property` : Convierte método en atributo (acceso sin paréntesis)
- `os.getenv(key, default)` : Busca en env vars, si no existe usa default
- Precedencia: `.env` > `config.yaml`

```
settings = Settings()
```

- Patrón Singleton: Una sola instancia compartida
- Evita recargar config en cada import

3.4 app/models/schemas.py

Explicación del Código:

```
class APIMetadata(BaseModel):  
    ingestion_date: datetime = Field(default_factory=datetime.now)
```

- `Field(default_factory=...)`: Genera valor por defecto dinámicamente
- `datetime.now` se ejecuta al crear instancia (no al definir clase)

```
error_message: Optional[str] = None
```

- `Optional[str]`: Acepta string o None
- Equivalente a `Union[str, None]`

```
class BronzeRecord(BaseModel):  
    metadata: APIMetadata  
    data: Dict[str, Any]
```

- Validación anidada: Pydantic valida `APIMetadata` automáticamente
- `Dict[str, Any]`: Dict con keys string, values cualquier tipo

```
record.model_dump(mode='json')
```

- `mode='json'`: Convierte datetime a ISO string
- Resultado es JSON-serializable (para `json.dumps()`)

3.5 app/models/audit_schemas.py

Explicación del Código:

```
class ExecutionStatus(str, Enum):  
    SUCCESS = "success"
```

- Hereda de `str` y `Enum`: Comportamiento de string con validación enum
- Permite comparación: `status == "success"` o `status == ExecutionStatus.SUCCESS`

```
audit_id: str = Field(default_factory=lambda: datetime.now().strftime("%Y%m%d_%H%M%S_%f"))
```

- `lambda`: Función anónima para generar timestamp

- `%f`: Microsegundos (unidad en timestamps rápidos)

```
metadata: Dict[str, Any] = Field(default_factory=dict)
```

- `default_factory=dict`: Crea dict vacío nuevo por instancia
- NO usar `metadata: Dict = {}` (compartiría mismo dict entre instancias)

```
quality_checks: List[DataQualityCheck] = Field(default_factory=list)
```

- Lista mutable con factory para evitar shared state

NIVEL 2: AUDITORÍA

3.6 `app/audit/audit_logger.py`

Explicación del Código:

```
def _ensure_audit_structure(self):
    directories = [
        self.audit_path / "executions",
        self.audit_path / "quality_checks",
        self.audit_path / "errors",
        self.audit_path / "metrics"
    ]
    for directory in directories:
        directory.mkdir(parents=True, exist_ok=True)
```

- `parents=True`: Crea directorios padres si no existen
- `exist_ok=True`: No lanza error si directorio ya existe
- Garantiza estructura antes de escribir

```
def log_execution(self, audit_log: AuditLog) -> Path:
    date_partition = datetime.now().strftime("%Y/%m/%d")
    output_dir = self.audit_path / "executions" / date_partition
```

- Particionamiento por fecha: Facilita queries temporales
- Ejemplo path: `audit/executions/2026/05/01/`

```
with open(file_path, 'w', encoding='utf-8') as f:
    json.dump(audit_log.model_dump(mode='json'), f, indent=2, ensure_ascii=False)
```

- `encoding='utf-8'`: Soporte caracteres especiales
- `indent=2`: JSON legible (pretty-print)
- `ensure_ascii=False`: Preserva caracteres UTF-8 (ñ, á, etc.)
- `model_dump(mode='json')`: Serializa Pydantic a dict JSON-compatible

```
def get_execution_summary(self, date: datetime = None):
    if date is None:
        date = datetime.now()
```

- Default a fecha actual si no se especifica

```
for file in execution_dir.glob("*.json"):
    with open(file, 'r') as f:
        logs.append(json.load(f))
```

- `glob("*.json")`: Busca todos los archivos JSON en directorio
- Lee y parsea cada archivo a dict

```
"successful": sum(1 for log in logs if log['status'] == 'success')
```

- Generator expression: Suma 1 por cada log con status success
- Eficiente en memoria (no crea lista intermedia)

3.7 app/audit/control_manager.py

Explicación del Código:

```
self.current_execution: Optional[ExecutionControl] = None
```

- Atributo de estado: Mantiene ejecución activa
- `Optional`: Puede ser None si no hay ejecución activa

```
def start_execution(self, pipeline_name: str, input_parameters: Dict[str, Any]):
    self.current_execution = ExecutionControl(
        pipeline_name=pipeline_name,
        input_parameters=input_parameters
    )
```

- Crea objeto ExecutionControl con Pydantic
- `execution_id` y `start_time` se generan automáticamente (default_factory)

```
def end_execution(self, status: ExecutionStatus, output_summary: Dict[str, Any]):
    if not self.current_execution:
        raise ValueError("No active execution")
```

- Guard clause: Previene finalizar sin ejecución activa

```
duration = (self.current_execution.end_time -
            self.current_execution.start_time).total_seconds()
```

- Resta de datetime objects devuelve timedelta

- `total_seconds()`: Convierte timedelta a float

```
audit_log = AuditLog(
    pipeline_name=self.current_execution.pipeline_name,
    execution_id=self.current_execution.execution_id,
    status=status,
    duration_seconds=duration,
    records_processed=output_summary.get('total_series', 0),
    records_success=output_summary.get('successful', 0),
    records_failed=output_summary.get('failed', 0),
    metadata=self.current_execution.input_parameters
)
```

- Construcción de AuditLog con datos de ejecución
- `.get(key, default)`: Retorna default si key no existe

```
def perform_quality_check(self, check_name, check_type, dataset, data, validation_func):
    validation_results = [validation_func(record) for record in data]
```

- List comprehension: Aplica función validación a cada record
- `validation_func`: Callable que retorna bool

```
records_passed = sum(validation_results)
```

- `sum()` en lista de bools: True=1, False=0
- Cuenta cuántos True hay

```
failure_rate = records_failed / records_checked if records_checked > 0 else 0
```

- Ternary operator: Previene división por cero
- Retorna 0 si no hay records

```
if failure_rate == 0:
    status = DataQualityStatus.PASSED
elif failure_rate < 0.05:
    status = DataQualityStatus.WARNING
else:
    status = DataQualityStatus.FAILED
```

- Lógica de umbrales: 0% = PASSED, <5% = WARNING, >=5% = FAILED
 - Threshold configurable según requerimientos
-

NIVEL 3: I/O

3.8 app/clients/base_client.py

Explicación del Código:

```
class BaseAPIClient(ABC):
```

- `ABC`: Abstract Base Class
- No se puede instanciar directamente, solo heredar

```
@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=2, min=2, max=10)
)
def _make_request(self, endpoint, method="GET", params=None):
```

- Decorator `@retry`: De librería tenacity
- `stop_after_attempt(3)`: Máximo 3 intentos
- `wait_exponential(multiplier=2, min=2, max=10)`: Backoff exponencial
 - Intento 1: Sin espera
 - Intento 2: Espera 2s
 - Intento 3: Espera 4s
 - Max 10s de espera

```
url = f"{self.base_url}/{endpoint}"
```

- f-string: Interpolación de variables en string
- Construye URL completa

```
response = requests.request(
    method=method,
    url=url,
    params=params,
    timeout=self.timeout
)
```

- `requests.request()`: Método genérico (GET, POST, etc.)
- `timeout`: Previene requests colgados indefinidamente

```
response.raise_for_status()
```

- Lanza excepción si status code 4xx o 5xx
- Manejo automático de errores HTTP

```
return response.json()
```

- Parsea response body como JSON a dict Python

```
except requests.exceptions.RequestException as e:  
    self.logger.error("API request failed", error=str(e), url=url)  
    raise
```

- Captura todas las excepciones de requests
- Logea error pero re-lanza excepción para manejo upstream

```
@abstractmethod  
def fetch_data(self, **kwargs) -> Dict[str, Any]:  
    pass
```

- Método abstracto: Subclases DEBEN implementarlo
- `**kwargs`: Acepta cualquier argumento nombrado

3.9 app/clients/bcr_client.py

Explicación del Código:

```
class BCRClient(BaseAPIClient):
```

- Hereda de BaseAPIClient
- Obtiene retry logic y logging automáticamente

```
def fetch_data(self, series_code, start_date=None, end_date=None, format="json"):  
    if not start_date or not end_date:  
        start_date, end_date = get_date_range(days_back=30)
```

- Guard clause: Si no hay fechas, usa últimos 30 días
- `not` evalúa None como False

```
endpoint = f"{series_code}/{format}/{start_date}/{end_date}"
```

- Construye endpoint según formato API BCR
- Ejemplo: "PD04637PD/json/2026-4-1/2026-4-30"

```
return self._make_request(endpoint)
```

- Llama método heredado
- Retry logic se aplica automáticamente


```
def fetch_multiple_series(self, series_codes, start_date=None, end_date=None,
format="json"):
    codes = "-".join(series_codes)
```

- `"-".join()`: Une lista con guion
- `["PD04637PD", "PD04638PD"] ---> "PD04637PD-PD04638PD"`
- API BCR permite múltiples series en una request

3.10 `app/storage/data_lake.py`

Explicación del Código:

```
def write_bronze(self, record, source, endpoint, partition_date=None):
    if partition_date is None:
        partition_date = datetime.now()
```

- Default a fecha actual para particionamiento

```
partition_path = partition_date.strftime("%Y/%m/%d")
```

- Formato: "2026/05/01" (con ceros)

```
output_dir = (
    self.base_path /
    "bronze" /
    source /
    endpoint /
    partition_path
)
```

- Operador `/`: Pathlib une paths elegantemente
- Resultado: `Path("data/bronze/bcr/exchange_rate/2026/05/01")`

```
output_dir.mkdir(parents=True, exist_ok=True)
```

- Crea toda la jerarquía de directorios si no existe

```
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S_%f")
file_path = output_dir / f"{timestamp}.json"
```

- `%f`: Microsegundos para unicidad
- Ejemplo: "20260501_103045_123456.json"

```
with open(file_path, 'w', encoding='utf-8') as f:
    json.dump(record.model_dump(mode='json'), f, indent=2, ensure_ascii=False)
```

- Context manager `with`: Cierra archivo automáticamente
- `record.model_dump(mode='json')`: Serializa Pydantic model
- `indent=2`: Pretty-print para legibilidad
- `ensure_ascii=False`: Preserva caracteres UTF-8

NIVEL 4: SERVICIOS

3.11 `app/services/ingestion_service.py`

Explicación del Código:

```
def __init__(self, client, data_lake, control_manager=None):
    self.client = client
    self.data_lake = data_lake
    self.control_manager = control_manager
```

- Dependency Injection: Recibe instancias configuradas
- `control_manager=None`: Auditoría es opcional

```
def ingest_series(self, series_code, start_date=None, end_date=None):
    start_time = time.time()
```

- Marca timestamp inicio para calcular duración

```
try:
    data = self.client.fetch_data(
        series_code=series_code,
        start_date=start_date,
        end_date=end_date
    )
```

- Try-catch para manejo de errores de red/API

```
if self.control_manager:
    self._validate_data_quality(series_code, data)
```

- Guard: Solo valida si hay control_manager
- Permite usar servicio sin auditoría

```
execution_time = (time.time() - start_time) * 1000
```

- Calcula duración en milisegundos
- `time.time()` retorna segundos desde epoch

```

metadata = APIMetadata(
    source="bcr",
    endpoint=f"exchange_rate/{series_code}",
    ingestion_date=datetime.now(),
    status="success",
    execution_time_ms=execution_time
)

```

- Construye metadata con info de ingesta
- Pydantic valida tipos automáticamente

```

record = BronzeRecord(metadata=metadata, data=data)

```

- Empaqueta metadata + data en BronzeRecord
- `data`: JSON raw del API (sin transformar)

```

file_path = self.data_lake.write_bronze(
    record=record,
    source="bcr",
    endpoint="exchange_rate"
)

```

- Persiste en Data Lake con particionamiento

```

return {
    "status": "success",
    "series_code": series_code,
    "file_path": str(file_path),
    "execution_time_ms": execution_time,
    "records_count": len(data.get('periods', []))
}

```

- Retorna dict con resumen de ingesta
- `.get('periods', [])`: Default a lista vacía si no existe

```

except Exception as e:
    self.logger.error("Series ingestion failed", series_code=series_code, error=str(e))

```

- Captura cualquier excepción
- Loggea con contexto (series_code, error)

```

if self.control_manager:
    self.control_manager.log_pipeline_error(
        error_type="IngestionError",
        error_message=str(e),
        context={"series_code": series_code}
    )

```

- Registra error en sistema de auditoría
- Permite análisis post-mortem

```
def _validate_data_quality(self, series_code, data):
    def validate_record(record: Dict) -> bool:
        return (
            'name' in record and
            'values' in record and
            len(record['values']) > 0 and
            record['values'][0] is not None
        )
```

- Función anidada: Scope local a método
- Valida estructura esperada de cada period
- Retorna bool: True si válido, False si no

```
periods = data.get('periods', [])

if periods:
    self.control_manager.perform_quality_check(
        check_name=f"completeness_{series_code}",
        check_type="completeness",
        dataset=series_code,
        data=periods,
        validation_func=validate_record
    )
```

- Solo ejecuta check si hay periods
- `validation_func`: Pasa función como parámetro
- ControlManager aplica función a cada record

NIVEL 5: ORQUESTACIÓN

3.12 `app/pipelines/bronze/bronze_pipeline.py`

Explicación del Código:

```
def __init__(self):
    self.logger = StructuredLogger(self.__class__.__name__)
```

- `self.__class__.__name__`: Obtiene nombre de clase dinámicamente
- Logger se identifica como "BronzePipeline"

```
self.client = BCRCClient(base_url=settings.bcr_base_url)
self.data_lake = DataLake(base_path=settings.data_lake_path)
```

- Inicializa componentes con configuración centralizada
- Dependency Injection manual

```
audit_path = settings.data_lake_path / "audit"
self.audit_logger = AuditLogger(audit_path=audit_path)
self.control_manager = ControlManager(audit_logger=self.audit_logger)
```

- Construye sistema de auditoría
- ControlManager depende de AuditLogger

```
self.ingestion_service = IngestionService(
    self.client,
    self.data_lake,
    self.control_manager
)
```

- Inyecta todas las dependencias al servicio

```
def run(self, series_codes, start_date=None, end_date=None):
    execution = self.control_manager.start_execution(
        pipeline_name="bronze_exchange_rate",
        input_parameters={
            "series_codes": series_codes,
            "start_date": start_date,
            "end_date": end_date
        }
    )
```

- Inicia control de ejecución con parámetros
- Genera execution_id único y timestamp

```
self.logger.info("Bronze pipeline started", execution_id=execution.execution_id)
```

- Log estructurado con contexto

```
results = []
errors = []

for code in series_codes:
    result = self.ingestion_service.ingest_series(
        series_code=code,
        start_date=start_date,
        end_date=end_date
    )
    results.append(result)

    if result['status'] == 'error':
        errors.append({
            "series_code": code,
```

```

        "error": result.get('error')
    })

```

- Loop secuencial por serie
- Acumula resultados y errores
- Un error no detiene el resto (resiliencia)

```

success_count = sum(1 for r in results if r['status'] == 'success')

```

- Generator expression: Cuenta exitosos eficientemente

```

if success_count == len(results):
    final_status = ExecutionStatus.SUCCESS
elif success_count == 0:
    final_status = ExecutionStatus.FAILED
else:
    final_status = ExecutionStatus.PARTIAL

```

- Lógica de status:
 - 100% exitoso ---> SUCCESS
 - 0% exitoso ---> FAILED
 - Mixto ---> PARTIAL

```

output_summary = {
    "total_series": len(results),
    "successful": success_count,
    "failed": len(results) - success_count,
    "errors": errors,
    "details": results
}

```

- Construye resumen con métricas
- `details`: Array completo de resultados individuales

```

self.control_manager.end_execution(
    status=final_status,
    output_summary=output_summary
)

```

- Finaliza control: Calcula duración, crea AuditLog, persiste

```

execution_report = self.control_manager.get_execution_report()

```

- Obtiene reporte completo con quality checks

```
return {  
    "pipeline": "bronze",  
    "execution_id": execution.execution_id,  
    **output_summary,  
    "execution_report": execution_report  
}
```

- `**output_summary`: Unpacking dict (spread operator)
- Retorna dict con toda la info de ejecución